CPNCheck

Tratamento da explosão de estados

redução por ordem parcial com conjuntos teimosos (*stubborn sets*)

Documentação técnica

5 de outubro de 2026

Sobre este documento. Ele descreve a etapa do CPNCheck que combate a explosão de estados: de onde a explosão vem nas redes geradas pelo pyRuleAnalyzer, quais técnicas da literatura foram avaliadas (inclusive as três do capítulo 8 de Jensen & Kristensen), por que a escolhida foi a redução por ordem parcial com *conjuntos teimosos*, como ela funciona passo a passo, o que exatamente ela garante, onde ela para de funcionar, e como foi validada.

Não há código aqui. O texto explica o *funcionamento* e o *porquê*; a implementação está em `cpncheck/reduction.py` e os testes em `tests/test_reduction.py`.

Sumário

1 O problema: de onde vem a explosão **4**
1.1 O que o model checker enumera 4
1.2 O que foi medido nas redes do pyRuleAnalyzer 4

2 As técnicas avaliadas **6**
2.1 Sweep-line 6
2.2 Simetria 6
2.3 Equivalência 7
2.4 O que sobra: redução por ordem parcial 7

3 A ideia central: independência **8**
3.1 O diamante 8
3.2 Do par ao conjunto 8

4 Dependência lida dos arcos **9**
4.1 Tirar, dar, testar 9
4.2 As relações derivadas 9

5 O conjunto teimoso de uma marcação **10**
5.1 As duas regras de fechamento 10
5.2 O que as regras garantem 10
5.3 Escolhendo entre candidatos 11
5.4 Exemplo: a floresta de três árvores, passo a passo 11

6 O que se preserva depende da pergunta **12**
6.1 Grafo *deadlock*: só D0–D2 12
6.2 Grafo *safety*: D0–D2 mais um objetivo 12
6.3 Grafo *stutter*: visibilidade, singleton e ciclos 13
6.4 Grafo *full* 14
6.5 Por que *dead* é tratado à parte 14

7 Da propriedade ao grafo **15**
7.1 Divisão em partes 15
7.2 Classificação 15
7.3 Caches 16

8 Detalhes que importam para a correção **17**
8.1 Habilitação num grafo reduzido 17
8.2 O não-determinismo continua visível 17
8.3 Contraexemplos 17
8.4 Orçamento 17

8.5 O relatório 18

9 O limite: visibilidade **19**

10 Validação **20**
10.1 Concordância de vereditos 20
10.2 Propriedades dos próprios grafos 20
10.3 O modo padrão não mudou 20

11 Resultados **21**

12 Uso e limitações **23**
12.1 Como ativar 23
12.2 Quando usar 23
12.3 Limitações 23

Referências **24**

Capítulo 1

O problema: de onde vem a explosão

1.1 O que o model checker enumera

O CPNCheck decide propriedades temporais ("a rede sempre termina", "a predição nunca recebe dois tokens", "toda execução chega a uma decisão") construindo o *grafo de ocorrência*: um nó para cada marcação alcançável a partir da marcação inicial, um arco para cada ocorrência de um elemento de ligação (uma transição com valores para suas variáveis). As fórmulas CTL são então avaliadas sobre esse grafo por pontos fixos.

O custo de tudo é o tamanho desse grafo. Se ele tem milhões de nós, cada propriedade custa milhões de passos; se tem 10^{30}, nenhuma é decidida. Isso é a *explosão de estados*, a principal limitação dos métodos de espaço de estados (Jensen & Kristensen, 2009, cap. 8).

1.2 O que foi medido nas redes do pyRuleAnalyzer

Antes de escolher uma técnica, foi medido *o quê* explode. As redes geradas a partir de modelos do scikit-learn têm uma estrutura fixa (Figura 1.1): uma transição `Distribute` copia a amostra para a entrada de cada árvore; cada árvore é uma página cujas transições são as folhas, uma por região do espaço de atributos; cada folha deposita a saída da árvore num lugar próprio; uma transição de agregação (`Vote` na floresta, `Decide_k` no *boosting*) consome todas as saídas e escreve a classe em `Prediction`.

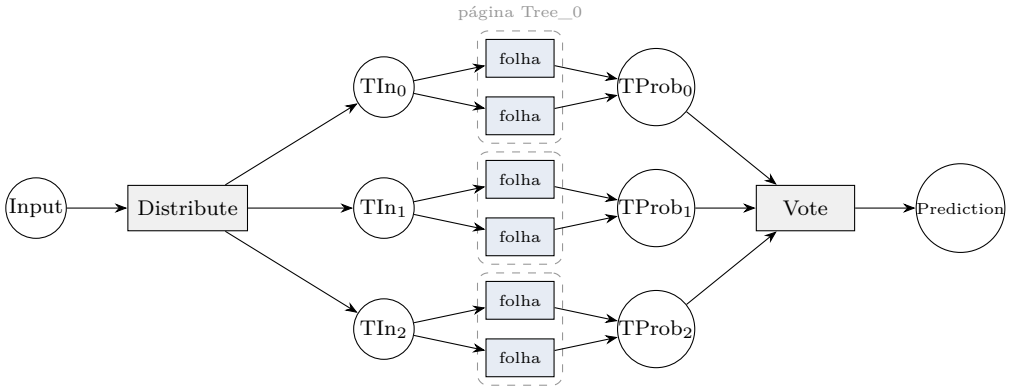


Figura 1.1: Uma floresta de três árvores como rede (simplificada: cada árvore real tem uma folha por região, com uma guarda sobre os atributos). As árvores não compartilham nenhum lugar entre `Distribute` e `Vote`.

Para uma amostra fixa, cada árvore tem exatamente uma folha habilitada, e as árvores são independentes entre si. Mesmo assim o grafo de ocorrência registra *todas as ordens* em que elas podem disparar. A Tabela 1.1 mostra o que foi medido.

Floresta aleatória, n árvores	1	2	3	4	5	6	7	8
Marcações alcançáveis	4	6	10	18	34	66	130	258

Boosting, 3 classes, M estágios	1	2	3	4	5
Marcações alcançáveis	218	731	1 730	3 377	5 834

Tabela 1.1: Tamanho do grafo de ocorrência, medido. A floresta tem exatamente $2^n + 2$ marcações; o *boosting* binário (um canal) é linear em M; o multiclasse é o produto dos tamanhos dos K canais, que andam em paralelo.

A fórmula da floresta é exata e conta a história inteira: depois de `Distribute`, cada marcação corresponde a um *subconjunto* das árvores que já votaram, e há 2^n subconjuntos; somam-se a marcação inicial e a final. A Figura 1.2 mostra o caso $n = 3$: um cubo.

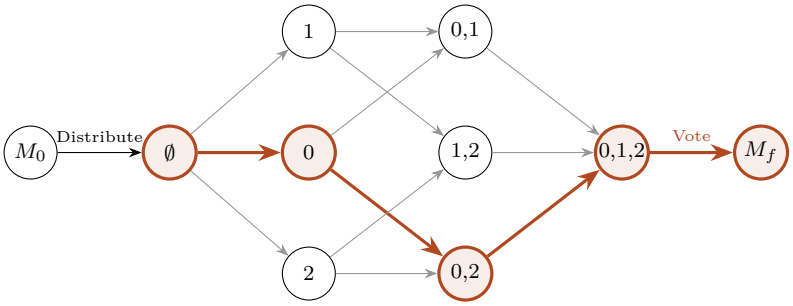


Figura 1.2: O grafo de ocorrência de uma floresta de três árvores. Cada nó do meio é o conjunto de árvores que já votaram. Em destaque, o único caminho que o grafo reduzido explora: $n + 3$ nós em vez de $2^n + 2$.

A conclusão da medição. A explosão nessas redes é *exclusivamente* entrelaçamento (*interleaving*) de componentes independentes. Nenhuma árvore tem mais de um caminho interno para uma amostra fixa; o que cresce é o número de ordens em que n coisas independentes podem acontecer. Uma floresta de 100 árvores tem $2^{100} + 2 \approx 1{,}3 \times 10^{30}$ marcações — e todas as ordens levam ao mesmo voto.

Capítulo 2

As técnicas avaliadas

O capítulo 8 de Jensen & Kristensen apresenta três métodos de redução desenvolvidos no contexto de CPN: *sweep-line*, simetria e equivalência. Ele próprio observa que nenhum método funciona para todo tipo de sistema, que cada um explora uma característica específica do modelo, e que o método escolhido precisa (i) explorar uma característica presente no sistema e (ii) preservar as propriedades que se quer verificar. Os três foram avaliados com esses dois critérios contra as redes medidas acima.

2.1 Sweep-line

Ideia. Se existe uma *medida de progresso* que nunca diminui ao longo das ocorrências, o grafo pode ser explorado camada por camada e as camadas já processadas podem ser apagadas da memória: nenhuma marcação futura volta a elas. O pico de memória cai para o tamanho de uma camada.

Aplica-se? Sim, e trivialmente: as redes são acíclicas, e o número de transições já disparadas é uma medida de progresso monotônica.

Ajuda? Pouco. A camada mais larga da floresta é a do meio do cubo, com $\binom{n}{\lfloor n/2 \rfloor}$ marcações — para $n = 8$, o pico medido foi 126 de 258 nós, um ganho de cerca de $2\times$, o mesmo patamar da tabela de estatísticas do próprio capítulo. Para $n = 100$ o pico ainda é da ordem de 10^{29}. A técnica reduz memória, não o número de estados visitados.

Preserva? Só propriedades de segurança ("existe marcação alcançável que satisfaz um predicado"). Perderíamos as propriedades de vivacidade (`AF`), a de recuperabilidade (`AG EF`), a de *home marking* (que precisa dos componentes fortemente conexos) e os contraexemplos, porque a versão básica apaga justamente os caminhos que eles reproduzem.

2.2 Simetria

Ideia. Se componentes são intercambiáveis (os receptores idênticos do protocolo do capítulo), marcações que diferem só por uma permutação das identidades viram um único nó.

Aplica-se? Não. O critério de corretude exige que toda guarda e toda inscrição de arco seja simétrica sob a permutação. As árvores de uma floresta têm limiares diferentes; trocar a árvore i pela j não é uma simetria da rede. O mesmo vale para os canais do *boosting*. Só árvores literalmente idênticas seriam permutáveis, o que quase não ocorre com *bootstrap*.

2.3 Equivalência

Ideia. Generaliza a simetria: o usuário define diretamente quais marcações são equivalentes, com a obrigação de que marcações equivalentes tenham conjuntos equivalentes de elementos habilitados e de sucessores.

Aplica-se? Não foi encontrada relação útil que satisfaça o critério. Duas marcações com subconjuntos diferentes de árvores já disparadas habilitam transições diferentes; não são equivalentes. Além disso, a prova de consistência é manual — e o CPNCheck já removeu uma vez um modo composicional justamente por falta de prova de preservação.

2.4 O que sobra: redução por ordem parcial

O mesmo capítulo lembra que há métodos desenvolvidos *fora* do contexto de CPN e aplicáveis a qualquer sistema de transições. Entre eles está a família que ataca exatamente o fenômeno medido: a *redução por ordem parcial* (*partial-order reduction*), que explora a *independência* entre ocorrências para não enumerar ordens que não importam. Sua forma mais antiga e mais geral são os *conjuntos teimosos* de Valmari (1988, 1991). A Tabela 2.1 resume a avaliação.

Técnica	Explora	Nas redes do pyRuleAnalyzer	Propriedades
Sweep-line	progresso monotônico	aplicável; pico $\approx \binom{n}{n/2}$, ganho $\sim 2\times$	só segurança
Simetria	componentes permutáveis	**incorreta**: guardas diferentes	todas as invariantes por simetria
Equivalência	relação dada pelo usuário	sem relação válida; prova manual	as que a relação respeita
Ordem parcial	**independência**	**floresta:** $2^n + 2 \to n + 3$	dependem das condições (cap. 6)

Tabela 2.1: Resumo da avaliação.

Capítulo 3

A ideia central: independência

3.1 O diamante

Duas ocorrências a e b são *independentes* numa marcação M quando ambas estão habilitadas, disparar uma não desabilita a outra, e as duas ordens levam à mesma marcação. No grafo isso aparece como um losango (Figura 3.1).

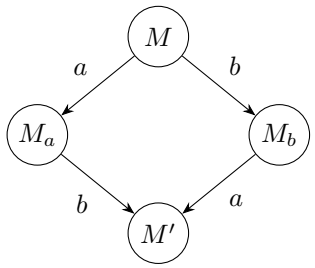


Figura 3.1: a e b independentes: as duas ordens chegam a M'. Explorar só uma delas não perde M'; perde apenas M_b (ou M_a), uma marcação intermediária.

A redução explora só um lado do losango. O que se perde é a marcação intermediária do outro lado. Toda a teoria que segue responde a uma pergunta: *quando* é seguro perder marcações intermediárias? A resposta depende do que se quer verificar. Se a pergunta é "a rede pode travar?", marcações intermediárias nunca importam, porque não são mortas. Se a pergunta é "a predição nunca tem dois tokens?", importam só as que podem violar isso. Se é "toda execução chega a uma decisão?", importa a ordem relativa de quem mexe na predição.

3.2 Do par ao conjunto

Com n árvores independentes, há $n!$ ordens das folhas e 2^n marcações intermediárias. Escolher sempre a mesma árvore primeiro — a de menor índice que ainda não votou — percorre um único caminho de n passos. Mas "independente" não pode ser um julgamento caso a caso sobre marcações já calculadas: a redução tem de decidir *antes* de explorar, olhando só a marcação atual e a estrutura da rede. É isso que o conjunto teimoso formaliza: um conjunto de transições, calculado na marcação atual, tal que disparar primeiro um de seus membros nunca impede nada que outra ordem poderia fazer.

Capítulo 4

Dependência lida dos arcos

O CPNCheck não tenta descobrir a independência executando a rede. Ele a lê da *estrutura*: de quais lugares cada transição tira tokens, em quais deposita, e quais apenas consulta. Isso é feito uma vez por rede.

4.1 Tirar, dar, testar

Para cada transição t e cada lugar p ligado a ela:

- t **tira** de p se há um arco de entrada de p para t;
- t **dá** a p se há um arco de saída de t para p;
- t **testa** p se há um arco de entrada e um de saída *com a mesma inscrição*. O token sai e volta igual: o efeito líquido é nulo, mas t precisa que ele esteja lá. Nesse caso p não conta como "tirado" nem "dado".

Os *lugares de entrada* de t são os que ela tira ou testa; os *lugares que t muda* são os que ela tira ou dá.

Por que estrutural e não semântica. Uma transição que dá a um lugar pode, para certos valores, não depositar nada (uma inscrição `if ... then 1‘x else empty`). A leitura estrutural a trata como "pode dar" sempre. Isso é uma *sobre-aproximação* da dependência: a redução pode manter mais estados do que precisaria, nunca menos. A correção não depende de entender o que as inscrições calculam.

4.2 As relações derivadas

A partir disso, para cada lugar p, ficam indexados: quem dá a p (*produtores*), quem tira de p, e quem precisa de p (tem p como lugar de entrada). E para cada transição t fica pronto o seu *conjunto de conflito*: todas as transições que mudam algum lugar de entrada de t, e todas as que precisam de algum lugar de que t tira. Na floresta, o conjunto de conflito de uma folha da árvore i é: as outras folhas da árvore i (que tiram de $\texttt{TIn}_i$ como ela) e `Distribute` (que dá a $\texttt{TIn}_i$). Nenhuma transição de outra árvore aparece — e isso é tudo o que a redução precisa saber para não entrelaçar as árvores.

Capítulo 5

O conjunto teimoso de uma marcação

5.1 As duas regras de fechamento

Em cada marcação M, o conjunto teimoso S começa com uma ou mais transições-semente e é fechado por duas regras, aplicadas até nada mudar:

1. **Transição habilitada em S.** Entram em S todas as que mudam algum lugar de entrada dela, e todas as que precisam de algum lugar de que ela tira — o seu conjunto de conflito.

   Efeito: nenhuma transição fora de S consegue desabilitá-la, nem ser desabilitada por ela, nem alterar os tokens dos quais suas ligações dependem.

2. **Transição desabilitada em S.** Entram os *habilitadores* dela. Se algum lugar de entrada tem menos tokens do que qualquer ligação exige, basta escolher *um* desses lugares e incluir seus produtores — o de menos produtores. Se nenhum está curto (a transição está desabilitada pelos *valores* dos tokens ou pela guarda), entram os produtores de todos os lugares de entrada.

   Efeito: nenhuma transição fora de S consegue habilitá-la. Tirar tokens nunca habilita nada, por isso só produtores importam.

Os elementos de ligação expandidos em M são então todos os elementos habilitados das transições de S — e só eles.

5.2 O que as regras garantem

As regras implementam as duas condições clássicas de Valmari:

D1 (comutação). Se uma sequência de transições fora de S seguida de uma transição $t \in S$ é possível a partir de M, então t seguida da mesma sequência também é, e chega à mesma marcação. Em outras palavras: o que está em S pode sempre ser antecipado.

D2 (chave). Existe em S uma transição habilitada que continua habilitada qualquer que seja a sequência de transições fora de S. Pela regra 1, *toda* transição habilitada de S é uma chave.

D0. S contém alguma transição habilitada sempre que M não é morta. O CPNCheck garante isso escolhendo pelo menos uma semente habilitada.

Uma sutileza de redes coloridas. Em redes P/T basta proteger a transição contra ser desabilitada. Em CPN, uma transição habilitada pode ganhar *novas ligações* quando alguém deposita tokens com outros valores nos seus lugares de entrada — e uma ligação nova que só existe depois de uma transição de fora de S quebraria D1. Por isso a regra 1 inclui em S também quem *dá* aos lugares de entrada, não apenas quem tira deles. Assim os tokens de entrada de t ficam intocados por tudo fora de S, e as ligações habilitadas de t são exatamente as mesmas antes e depois.

5.3 Escolhendo entre candidatos

Há em geral várias sementes possíveis, e cada uma gera um conjunto fechado diferente. O CPNCheck prefere o candidato que expande o menor número de elementos de ligação e para assim que encontra um com exatamente um. Como as duas regras olham uma transição por vez, o fecho de uma união é a união dos fechos; o fecho de cada transição isolada é calculado uma vez por marcação e combinado.

5.4 Exemplo: a floresta de três árvores, passo a passo

1. Em M_0 só `Distribute` está habilitada. Ela é expandida (não há o que reduzir).
2. Em $\emptyset$ estão habilitadas a folha certa de cada uma das três árvores. Semente: a folha da árvore 0. Regra 1: entram as outras folhas da árvore 0 e `Distribute`. As outras folhas estão desabilitadas (guarda falsa para esta amostra): regra 2, sem lugar curto, entram os produtores de $\texttt{TIn}_0$ — só `Distribute`, que já está. `Distribute` está desabilitada (`Input` vazio): regra 2, `Input` está curto e não tem produtores. Fim: S tem um único elemento habilitado. Só a árvore 0 dispara.
3. O mesmo acontece em $\{0\}$ com a árvore 1 e em $\{0, 1\}$ com a árvore 2.
4. Em $\{0, 1, 2\}$ só `Vote` está habilitada; dispara; a marcação final é morta.

O grafo reduzido é o caminho destacado na Figura 1.2: $n + 3$ nós. As marcações $\{1\}$, $\{2\}$, $\{0, 1\}\ldots$ nunca são construídas.

Capítulo 6

O que se preserva depende da pergunta

D0–D2 sozinhas garantem uma coisa: nenhuma marcação morta se perde. Isso basta para algumas propriedades e não para outras. O CPNCheck constrói até quatro *tipos* de grafo, cada um com as condições adicionais que a pergunta exige, e decide cada propriedade no tipo menos explorado que comprovadamente dá o mesmo veredito do grafo completo.

6.1 Grafo *deadlock*: só D0–D2

Teorema 6.1 (Valmari)**.** Com D0–D2, toda marcação morta alcançável no grafo completo é alcançável no grafo reduzido, e vice-versa (o reduzido é um subgrafo do completo).

Por quê, informalmente. Pegue um caminho do grafo completo até uma marcação morta. Ele precisa conter alguma transição de S: senão a chave de D2 continuaria habilitada até o fim, e a marcação não seria morta. Pela D1, a primeira transição de S no caminho pode ser antecipada; o reduzido dá esse passo, e o caminho restante é um passo mais curto. Por indução, o reduzido chega à mesma marcação morta.

O mesmo argumento mostra mais uma coisa, que o CPNCheck usa:

Teorema 6.2. Com D0–D2, existe uma sequência de ocorrência infinita no grafo completo se e somente se o grafo reduzido tem um ciclo.

Por quê. Dada uma sequência infinita, ou ela contém uma transição de S (que D1 antecipa), ou não contém nenhuma (e então a chave, que nunca é desabilitada, pode ser disparada antes de qualquer prefixo dela). Em ambos os casos o reduzido dá um passo e continua havendo uma sequência infinita a partir dali. Como o grafo é finito, isso fecha um ciclo.

Decide: terminação (`AF dead`), "toda marcação morta tem predição" (`AG(dead → pred)`) e, quando a rede termina, o *home marking*: num grafo finito acíclico os componentes terminais são as marcações mortas, e existe *home marking* exatamente quando há uma só.

6.2 Grafo *safety*: D0–D2 mais um objetivo

Para invariantes `AG` ψ com ψ proposicional ("a predição tem no máximo um token", "nunca há duas folhas da mesma árvore habilitadas"), a pergunta é se alguma marcação que viola ψ é alcançável — mesmo que não seja morta. Marcações intermediárias passam a importar, mas só as que podem violar ψ.

A condição extra vem de Kristensen & Valmari (2000): numa marcação em que ψ ainda vale, S precisa conter um *up-set* para $\neg\psi$ — um conjunto de transições tal que todo caminho que torna ψ falsa dispara alguma delas.

Teorema 6.3 (Kristensen & Valmari)**.** Com D0–D2 e o up-set, uma marcação violando ψ é alcançável no grafo completo se e somente se é alcançável no reduzido. Não é preciso condição de ciclo.

Por quê. Pegue o caminho mais curto até uma violação. Ele dispara alguma transição do up-set, e portanto de S; D1 antecipa a primeira; o caminho restante é estritamente mais curto. O comprimento cai a cada passo — não há como "andar em círculos" ignorando a violação.

Como o up-set é calculado

O up-set é calculado na marcação atual, a partir da fórmula e do seu valor presente. Para cada proposição atômica, a Tabela 6.1 diz quem consegue mudar seu valor.

Proposição	Hoje verdadeira; para ficar falsa	Hoje falsa; para ficar verdadeira
algum lugar marcado	tirar de *um* lugar marcado (qualquer um serve)	dar a algum dos lugares
todos marcados	tirar de algum dos lugares	dar a *um* lugar vazio
$\lvert p\rvert \le k$ para os lugares	dar a algum dos lugares	tirar de *um* lugar acima de k
tokens num domínio	dar a algum dos lugares	tirar de *um* lugar com token fora
T habilitada	tirar das entradas de *uma* habilitada	habilitadores de cada $t \in T$
$\lvert \mathrm{EN}(T)\rvert \le k$	habilitadores das desabilitadas, produtores das entradas das habilitadas	tirar das entradas de T

Tabela 6.1: Quem pode virar cada proposição. "*Um*" indica que basta escolher um elemento (o de menor conjunto), porque todos precisam mudar.

Os conectivos combinam isso pela lógica de "quem precisa mudar": uma conjunção verdadeira cai se *qualquer* conjunto cair, então os up-sets dos conjuntos se somam; uma conjunção falsa só fica verdadeira se *todo* conjunto falso virar, então basta o up-set de um deles, o menor. A disjunção é o dual, e a negação apenas inverte o valor.

Exemplo 6.1. Propriedade D1 da floresta: `AG(EN(Vote)` $\rightarrow$ `todas as árvores votaram)`. No início, `EN(Vote)` é falsa, e a implicação é verdadeira por causa dela. Para violá-la, `Vote` precisa ficar habilitada; o lugar de entrada curto de menor número de produtores é a saída de uma árvore; o up-set são as folhas daquela árvore. A busca segue linear.

6.3 Grafo *stutter*: visibilidade, singleton e ciclos

Para `AF`, `EF`, `EU` e combinações como `AG EF`, a *ordem* relativa dos eventos observados importa. A teoria aplicada é a de Gerth, Kuiper, Peled & Penczek (1999), que acrescenta a D0–D2:

C2 (visibilidade). Uma transição é *visível* se muda algum lugar que a fórmula observa (os lugares das proposições, ou os lugares de entrada das transições cuja habilitação a fórmula consulta). Um conjunto reduzido só pode conter transições invisíveis.

C3 (ciclo). Se o passo reduzido levaria a uma marcação que está na pilha da busca em profundidade (fecharia um ciclo), a marcação é expandida por completo. Sem isso, um ciclo poderia adiar para sempre uma transição que ninguém escolhe (o "problema da ignorância").

C4 (singleton). O conjunto reduzido tem *um único* elemento de ligação; caso contrário, expande-se tudo.

Teorema 6.4 (Gerth, Kuiper, Peled & Penczek)**.** Com essas condições o grafo reduzido é bissimilar por gagueira (*stuttering bisimilar*) ao completo em relação às proposições observadas. Portanto toda fórmula CTL sem o operador "próximo" (CTL-X) tem o mesmo valor nos dois.

Gagueira significa: as execuções do reduzido e do completo mostram as mesmas mudanças das proposições observadas, na mesma ordem, diferindo apenas em quantas vezes cada valor se repete. Como CTL-X não sabe contar passos, não distingue as duas. Nenhuma propriedade do catálogo usa "próximo".

Por causa de C3, este tipo é explorado em profundidade; os outros dois, em largura, o que dá contraexemplos mais curtos.

6.4 Grafo *full*

O grafo de ocorrência comum. Usado para as fórmulas que nenhum dos teoremas cobre — em particular as que mencionam marcações mortas de um jeito diferente dos formatos reconhecidos (por exemplo `AG EF dead`), e o *home marking* de uma rede que não termina.

6.5 Por que *dead* é tratado à parte

"A marcação é morta" não é uma proposição sobre lugares: *qualquer* transição pode torná-la verdadeira ao ser a última. Tratá-la como observada tornaria toda transição visível e anularia a redução. Por isso ela só aparece nos formatos que o grafo *deadlock* decide sem visibilidade: `AF dead`, `AG(dead` $\rightarrow \psi$`)`, `EF(dead` $\wedge\chi$`)` e a negação deste, com ψ, χ sem *dead*.

Capítulo 7

Da propriedade ao grafo

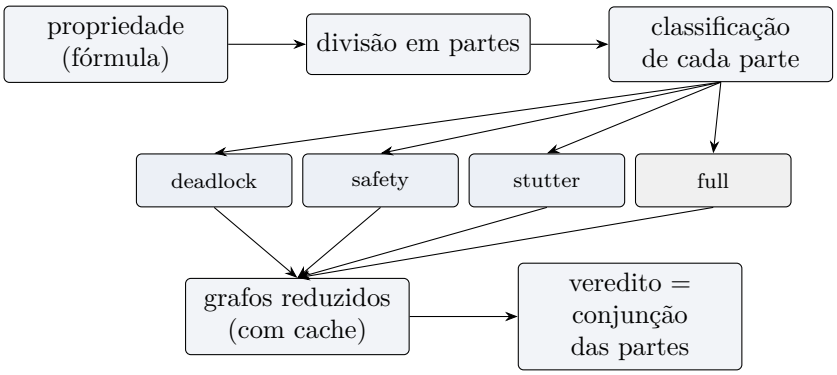


Figura 7.1: Cada propriedade é dividida, cada parte é classificada, e cada classe é decidida no seu grafo.

7.1 Divisão em partes

Uma parte que observa menos lugares tem menos transições visíveis e um up-set menor, e portanto reduz mais. Antes de classificar, toda propriedade é dividida em partes independentes cuja conjunção é ela mesma:

- **quantificador "para toda árvore/canal"**: vira uma parte por árvore ou canal, cada uma com o nome do membro, que aparece no contraexemplo;
- **`AG` de conjunção**: $\texttt{AG}(a \wedge b) = \texttt{AG}\, a \wedge \texttt{AG}\, b$;
- **limite sobre muitos lugares**: "todo lugar tem no máximo um token" (A7) vira uma invariante por lugar. Sem isso, o up-set seria "todos os produtores de todos os lugares" — isto é, tudo — e nada seria reduzido.

7.2 Classificação

Cada parte é examinada pela forma da fórmula: `AF dead` e as formas com *dead* reconhecidas vão para *deadlock*; `AG`, `!EF` e `EF` de fórmulas proposicionais sem *dead* vão para *safety* (com o ψ cuja violação é o objetivo); o resto sem *dead* vai para *stutter* (com o seu conjunto de transições visíveis); o que sobrar vai para *full*. A Tabela 7.1 mostra o resultado para o catálogo das redes de árvores.

Id	Propriedade	Grafo	Partes (RF, 100 árvores)
A1	terminação	deadlock	1
A2	sem travamento espúrio	deadlock	1
A3	decisão inevitável (`AF pred`)	stutter	1
A4	recuperabilidade (`AG EF pred`)	stutter	1
A5	saída única	safety	1
A6	rótulo válido	safety	1
A7	*safeness*	safety	202 (uma por lugar)
A8	*home marking*	deadlock*	—
B1	determinismo das folhas	safety	100
B2	seleção inevitável de folha	stutter	100
B3	saída única por árvore	safety	100
C1, C2	precedência dos estágios (GBDT)	stutter	por canal
C3a	decisão inevitável (GBDT)	stutter	1
C3b	decisão determinística (GBDT)	safety	1
D1	votação completa	safety	1
D2a	agregação inevitável (`AF EN(Vote)`)	stutter	1 (**não reduz**)
D2b	agregação única	safety	1

Tabela 7.1: Onde cada propriedade do catálogo é decidida. *Se a rede termina; senão, o grafo completo. B4 (estrutural) e PC (teste de conformidade) não usam grafo e não mudam.

7.3 Caches

Grafos são construídos sob demanda e guardados por tipo e parâmetro: um único grafo *deadlock* serve A1, A2 e A8; grafos *stutter* com o mesmo conjunto visível são compartilhados. Além disso, as centenas de grafos *safety* de uma mesma entrada percorrem quase sempre as mesmas primeiras marcações em ordens diferentes, então os elementos habilitados de cada marcação são lembrados entre grafos. E dentro de um grafo, ao passar de uma marcação para a seguinte, só são reavaliadas as transições que precisam de algum lugar que a ocorrência mudou — as outras mantêm exatamente as mesmas ligações.

Capítulo 8

Detalhes que importam para a correção

8.1 Habilitação num grafo reduzido

Propriedades como "nunca há duas folhas habilitadas" ou "`Vote` acaba habilitada" consultam a *habilitação* numa marcação. No grafo completo, habilitação e arcos de saída coincidem. No reduzido, não: os arcos são só os elementos expandidos. Por isso cada nó de um grafo reduzido guarda a lista completa de elementos habilitados encontrada durante a busca, e as proposições de habilitação são avaliadas sobre ela, nunca sobre os arcos. Sem isso, uma redução esconderia exatamente o não-determinismo que B1 procura.

8.2 O não-determinismo continua visível

Se uma rede mutante tem duas folhas da mesma árvore habilitadas ao mesmo tempo (uma sobreposição de guardas alcançada pela amostra), as duas tiram do mesmo lugar; pela regra 1 entram juntas em qualquer conjunto teimoso. A redução nunca escolhe uma e esconde a outra. Isso aparece nos testes: os mutantes de sobreposição continuam reprovados com a redução ativa.

8.3 Contraexemplos

Todo grafo reduzido é um subgrafo do completo, então todo caminho nele é uma sequência de ocorrência real da rede. Os contraexemplos encontrados num grafo reduzido são construídos e certificados exatamente como antes: o *replay*, que redispara a sequência com a regra de disparo da rede e confere a marcação final, continua sendo a prova de que não foram inventados. Eles podem não ser os mais curtos do grafo completo, apenas válidos.

8.4 Orçamento

Cada grafo tem o mesmo limite de marcações que o modo padrão. Uma parte cujo grafo estoura o limite fica *sem veredito*; a propriedade inteira fica sem veredito se nenhuma outra parte falhou, e o relatório diz qual grafo estourou e por quê. Um veredito nunca é chutado.

8.5 O relatório

No modo reduzido, o relatório mostra quantos grafos foram construídos, de quais tipos, quantos nós no total e o maior; o tamanho do grafo *deadlock*; e, abaixo de cada propriedade, em qual tipo de grafo ela foi decidida (por exemplo “safety x202”). Como o CPN Tools não tem essa redução, os números esperados do oráculo (nós, arcos, componentes) não são produzidos nesse modo: a comparação com o CPN Tools continua sendo feita no modo padrão.

Capítulo 9

O limite: visibilidade

Uma propriedade do catálogo não reduz: D2a, `AF EN(Vote)`, "a agregação acaba habilitada". A habilitação de `Vote` depende dos lugares de saída de *todas* as árvores, e toda folha dá a um deles. Logo toda folha é visível, nenhum conjunto reduzido satisfaz C2, e o grafo *stutter* desta propriedade é o grafo completo. Até 17 árvores ($2^{17} + 2 = 131\,074$ marcações) isso ainda cabe no orçamento padrão; a partir daí D2a fica sem veredito, com a explicação "k de m transições mudam o que a propriedade observa e não podem ser reordenadas".

Isso não é um defeito da implementação: é o limite conhecido da redução por ordem parcial para propriedades de vivacidade. A condição estática de visibilidade não sabe que só a *última* folha muda o valor da proposição — isso depende de quais árvores já votaram, informação que só a exploração tem. Duas observações tornam o limite pouco grave na prática:

- A3 (`AF pred`, "toda execução chega a uma decisão") observa só `Prediction`, é decidida em grafo de $n+3$ nós e já garante o que D2a garante para uma rede que produz predição apenas via `Vote`.
- D1 ("`Vote` só habilita quando todas votaram"), A1 e A2 cobrem o lado de segurança da agregação, todos reduzidos.

Possível continuação. O capítulo 8 observa que métodos de redução podem ser combinados. Para D2a, a combinação natural é com o *sweep-line*: o grafo completo dessa propriedade é acíclico e tem medida de progresso trivial, então poderia ser percorrido camada a camada com memória proporcional à camada mais larga. Isso troca o limite de memória por tempo, sem mudar o número de estados; não foi implementado.

Capítulo 10

Validação

Uma redução só vale se for invisível nas respostas. O critério adotado foi o mais forte disponível sem um assistente de provas: comparar veredito por veredito com o grafo completo, em redes construídas para falhar cada propriedade.

10.1 Concordância de vereditos

- **No CPNCheck**: as duas redes de referência (*boosting* binário e floresta multiclasse) e *todos* os seus mutantes — um por propriedade, incluindo ciclo, travamento, saída dupla, rótulo inválido, sobreposição de guardas, votação parcial, estágio pulado — somando 28 redes e **435 vereditos**: **nenhuma divergência**.
- **No pyRuleAnalyzer**: árvore de decisão, floresta, *boosting* binário e de três classes, nas versões inicial e refinada e em todos os mutantes de cada uma: mesma concordância total, e o caminho completo (*model_check* com redução) aprova as redes corretas.
- **Escala**: florestas de até 16 árvores e *boosting* de três classes com até 5 estágios, comparados com o grafo completo: nenhuma divergência.

10.2 Propriedades dos próprios grafos

- todo arco de um grafo reduzido é um arco do grafo completo;
- as marcações mortas do grafo *deadlock* são exatamente as do completo, em todas as redes e mutantes;
- a floresta de n árvores tem $2^n + 2$ marcações no completo e $n + 3$ no reduzido;
- num grafo *stutter*, toda marcação expandida parcialmente expandiu exatamente um elemento, invisível;
- os contraexemplos encontrados com a redução reexecutam na rede.

10.3 O modo padrão não mudou

Os testes de concordância com o CPN Tools 4.0.1 (executado sem interface, via Access/CPN) continuam passando no modo padrão: a redução é um caminho paralelo, ativado só quando pedido.

Capítulo 11

Resultados

	Grafo completo		Com redução			
Modelo	nós	tempo	*deadlock*	maior grafo	tempo	decididas
RF, 2 árvores	6	0,00 s	5	6	0,00 s	todas
RF, 4	18	0,00 s	7	18	0,01 s	todas
RF, 8	258	0,06 s	11	258	0,03 s	todas
RF, 12	4 098	1,39 s	15	4 098	0,28 s	todas
RF, 16	65 538	36,2 s	19	65 538	5,4 s	todas
RF, 20	$\approx 10^6$	—	23	(orçamento)	19 s	menos D2a
RF, 50	$\approx 10^{15}$	—	53	(orçamento)	37 s	menos D2a
RF, 100	$\approx 10^{30}$	—	103	(orçamento)	70 s	menos D2a
GBDT 3 cl., 1 estágio	218	0,01 s	18	22	0,03 s	todas
GBDT 3 cl., 3	1 730	0,17 s	36	40	0,15 s	todas
GBDT 3 cl., 5	5 834	0,78 s	54	58	0,48 s	todas

Tabela 11.1: Uma amostra por rede, orçamento de 200 000 marcações por grafo. Nas florestas o "maior grafo" é sempre o de D2a, que não reduz; ele também domina o tempo. A6 não entra na contagem das florestas porque o domínio de rótulos não foi declarado nesse experimento.

Na floresta de 100 árvores, tirando D2a, **todas as outras 12 propriedades de grafo** (B4 é estrutural e não usa grafo) são decididas em cerca de 6 s, construindo 507 grafos reduzidos de no máximo 103 nós cada; A7 sozinha usa 202 grafos (um por lugar) e leva cerca de 2 s. Os 64 s restantes são D2a esgotando o orçamento. No *boosting* multiclasse, onde nenhuma propriedade observa todas as árvores, tudo reduz: o maior grafo de qualquer propriedade tem 58 nós contra 5 834 do completo.

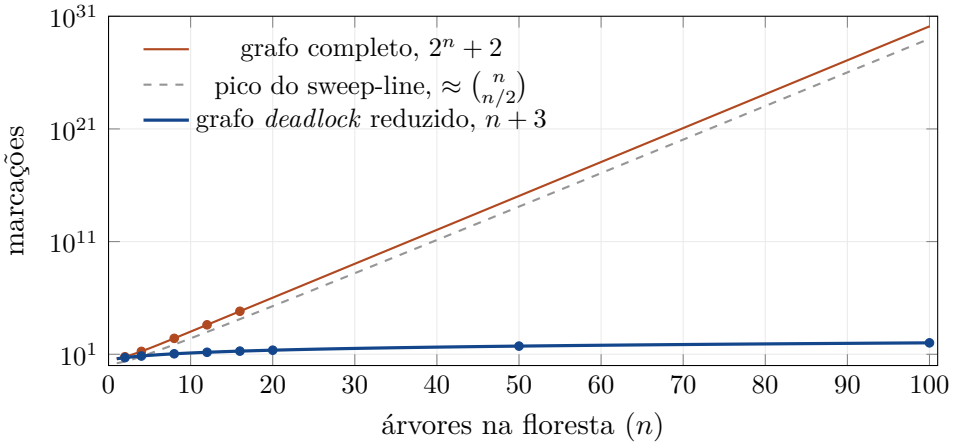


Figura 11.1: Crescimento em escala logarítmica. Pontos: medidos. O sweep-line acompanha o grafo completo a uma distância constante na escala log; a redução por ordem parcial muda a classe de crescimento.

Capítulo 12

Uso e limitações

12.1 Como ativar

A redução é opcional e o modo padrão não mudou. Ela é ativada por um parâmetro, `reduction`, com valor `"stubborn"`, aceito pela verificação do CPNCheck (na API e na linha de comando, como opção `--reduction stubborn`) e repassado pelo pyRuleAnalyzer na verificação do modelo e no *pipeline* verificado. Nenhuma propriedade, perfil ou rede precisa ser alterada: a divisão e a classificação são automáticas e valem para qualquer perfil, não só o das redes de árvores.

12.2 Quando usar

- **Florestas com mais de uns 12 árvores e *boosting* multiclasse**: é onde a explosão está, e onde a redução muda a classe de crescimento.
- **Comparação com o CPN Tools**: use o modo padrão; o oráculo compara também o tamanho do grafo, que a redução muda de propósito.
- **Redes pequenas**: tanto faz; a redução constrói vários grafos pequenos em vez de um, e pode não ser mais rápida.

12.3 Limitações

- **Visibilidade** (capítulo 9): propriedades de vivacidade que observam lugares mexidos por todos os componentes não reduzem.
- **Dependência estrutural**: a leitura dos arcos sobre-aproxima a dependência real; uma rede cujos componentes compartilham lugares "por precaução" reduz menos. Nunca reduz errado por isso.
- **Fórmulas com *dead*** fora dos formatos reconhecidos vão para o grafo completo.
- **Por entrada**: como no modo padrão, cada amostra na marcação inicial tem seus próprios grafos; a redução não generaliza vereditos de uma amostra para outra. Para "toda entrada", a análise estrutural das guardas (B4) continua sendo a ferramenta.

Referências

- C. Baier, J.-P. Katoen. *Principles of Model Checking*. MIT Press, 2008. Capítulo 8 (redução por ordem parcial).
- E. M. Clarke, O. Grumberg, D. Kroening, D. Peled, H. Veith. *Model Checking*, 2.ª ed. MIT Press, 2018.
- R. Gerth, R. Kuiper, D. Peled, W. Penczek. A partial order approach to branching time logic model checking. *Information and Computation* 150(2), 1999.
- P. Godefroid. *Partial-Order Methods for the Verification of Concurrent Systems*. LNCS 1032, Springer, 1996.
- K. Jensen, L. M. Kristensen. *Coloured Petri Nets: Modelling and Validation of Concurrent Systems*. Springer, 2009. Capítulo 8 (*Advanced State Space Methods*).
- L. M. Kristensen, A. Valmari. Improved question-guided stubborn set methods for state properties. *ICATPN 2000*, LNCS 1825, 2000.
- D. Peled. All from one, one for all: on model checking using representatives. *CAV 1993*, LNCS 697, 1993.
- A. Valmari. Stubborn sets for reduced state space generation. *Advances in Petri Nets 1990*, LNCS 483, 1991.
- A. Valmari, H. Hansen. Stubborn set intuition explained. *Transactions on Petri Nets and Other Models of Concurrency XII*, LNCS 10470, 2017.